

Informe — Autenticación contraseña + TOTP

Proyecto Final — IIC2531 Seguridad Computacional

Miguel Ulloa Wood · Alexander Burgos Quezada

22 de junio de 2026

- [Resumen](#)
- [1. Qué hicimos, problema y relevancia](#)
 - [1.1 Qué construimos](#)
 - [1.2 El problema de seguridad y por qué importa](#)
 - [1.3 Conexión con el curso](#)
- [2. Modelo de amenaza y superficie de ataque](#)
- [3. Diseño y arquitectura](#)
- [4. Implementación y decisiones de seguridad](#)
 - [4.1 Contraseñas: bcrypt](#)
 - [4.2 TOTP desde cero \(RFC 4226 / RFC 6238\)](#)
 - [4.3 Cifrado del secreto en reposo \(corrección de la propuesta\)](#)
 - [4.4 Ventana de tolerancia y anti-replay](#)
 - [4.5 Comparación en tiempo constante](#)
 - [4.6 Bloqueo por fuerza bruta \(lockout\)](#)
 - [4.7 Resistencia a enumeración de usuarios](#)
 - [4.8 Códigos de respaldo](#)
 - [4.9 Sesiones, CSRF y cabeceras](#)
- [5. Metodología de evaluación](#)
- [6. Resultados](#)
 - [6.1 E1 — Conformidad con los RFC \(correctitud e interoperabilidad\)](#)
 - [6.2 E4 — Fuerza bruta online vs. bloqueo](#)
 - [6.3 E3 — Canal lateral por tiempo](#)
 - [6.4 E5 — Replay](#)
 - [6.5 E6 — Pruebas automatizadas](#)
- [7. Cambios respecto de la propuesta y limitaciones](#)
 - [7.1 Cambios respecto de la propuesta original](#)
 - [7.2 Limitaciones y riesgo residual](#)
- [8. Consideraciones éticas](#)
- [9. Trabajo futuro](#)
- [10. Conclusión](#)
- [11. Cómo reproducir](#)

- [12. Referencias](#)

Resumen

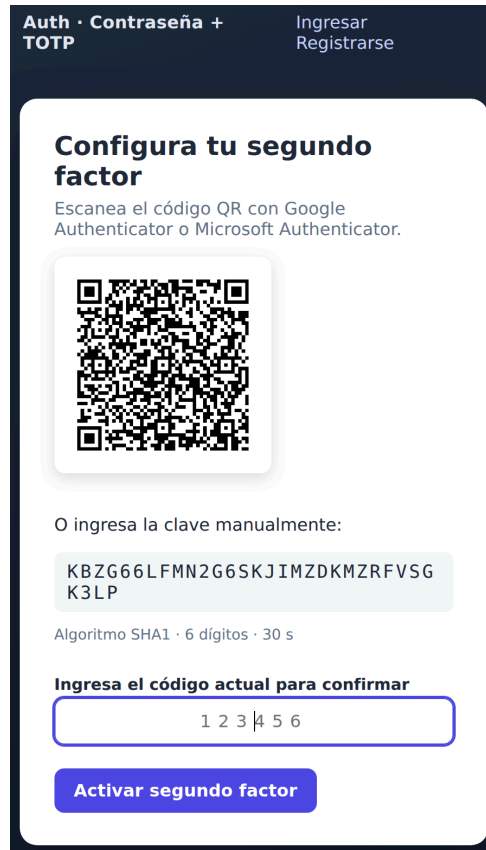
Construimos, desde cero y en software, un servidor local de autenticación de dos factores: **contraseña** (almacenada con bcrypt) más un **segundo factor TOTP** (contraseña de un solo uso basada en tiempo, RFC 6238) que interopera con Google Authenticator y Microsoft Authenticator. Más allá de implementar el “happy path”, tratamos el proyecto como un ejercicio de seguridad: definimos un **modelo de amenaza**, implementamos defensas concretas (cifrado del secreto en reposo, comparación en tiempo constante, bloqueo por fuerza bruta con *backoff*, anti-replay, protección CSRF, resistencia a enumeración de usuarios) y luego **atacamos nuestra propia implementación** con un arnés de experimentos para medir si las defensas funcionan. Los resultados son cuantitativos: 16/16 vectores oficiales RFC 4226/6238 correctos; el bloqueo reduce los intentos de adivinación de ~180.000/hora a ~12/hora (de ~3 horas a ~800 días para una probabilidad de éxito de 50%); la comparación con `hmac.compare_digest` elimina una fuga por tiempo de ~16 ns/carácter; y los códigos repetidos son rechazados. También corregimos un **error de seguridad de nuestra propia propuesta** (proponía *hashear* el secreto TOTP, lo cual es imposible de verificar) y lo reemplazamos por **cifrado autenticado**.

1. Qué hicimos, problema y relevancia

1.1 Qué construimos

Un servidor HTTP local (no desplegado) en **Python + Flask** que ofrece:

- Registro de cuentas con contraseña.
- Enrolamiento de un segundo factor TOTP mediante código QR (`otpauth://`).
- Login en **dos pasos**: contraseña y luego código TOTP de 6 dígitos.
- **Códigos de respaldo** de un solo uso para recuperación.
- Una interfaz web simple (login, QR, verificación, panel).



Pantalla de enrolamiento del segundo factor: QR `otpauth://`, clave en Base32 y confirmación del primer código.

El núcleo criptográfico (HOTP/TOTP) está implementado **desde cero** con la librería estándar (`hmac`, `hashlib`), no con una librería de alto nivel, para entender y controlar cada paso del algoritmo.

1.2 El problema de seguridad y por qué importa

Las contraseñas, por sí solas, son un factor débil: se reutilizan, se filtran en brechas masivas y son vulnerables a *phishing* y a adivinación. La autenticación de dos factores (2FA) mitiga gran parte de esto: aunque el atacante conozca la contraseña, necesita además el segundo factor. TOTP es el mecanismo 2FA más extendido porque no requiere infraestructura adicional (SMS, hardware): solo un secreto compartido y un reloj.

Sin embargo, **implementar 2FA correctamente es sutil**. Un detalle mal resuelto —comparar códigos con `==`, no limitar los intentos, guardar el secreto en claro, no prevenir la repetición— rompe la garantía de seguridad. Este proyecto explora precisamente esos detalles: no basta con que “funcione”, debe resistir ataques.

1.3 Conexión con el curso

El proyecto integra varios temas centrales de Seguridad Computacional: **criptografía aplicada** (HMAC, hashing de contraseñas, cifrado autenticado), **autenticación y autorización**, **modelo de amenaza y superficie de ataque**, **canales laterales** (timing), **defensa en profundidad**, **privilegio mínimo** (el secreto se descifra solo cuando se necesita; la clave maestra vive fuera de la BD) y **seguridad web** (CSRF, cabeceras, manejo de sesión, inyección SQL).

2. Modelo de amenaza y superficie de ataque



Modelo de amenaza y fronteras de confianza

Activos a proteger: las contraseñas, los secretos TOTP y la integridad de las sesiones.

Atacante considerado (en alcance): un atacante remoto sin credenciales que interactúa con el servidor a través de HTTP, capaz de (a) intentar adivinar contraseñas y códigos, (b) observar tiempos de respuesta, (c) reenviar peticiones (replay/CSRF), y un atacante que (d) obtiene una **copia del archivo de base de datos** (p. ej. por un respaldo mal protegido o una inyección de lectura).

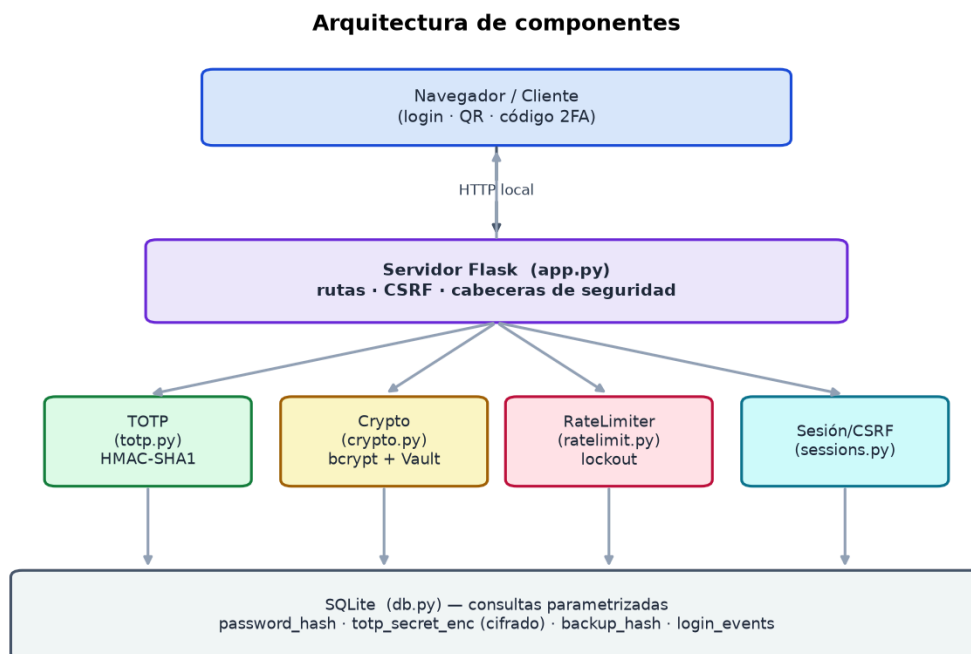
Defensas correspondientes (resumen):

Amenaza	Defensa implementada
Fuerza bruta de contraseña/código	Bloqueo con <i>backoff</i> exponencial + throttle por IP

Amenaza	Defensa implementada
Fuga del archivo <code>.db</code>	Contraseñas con bcrypt; secreto TOTP cifrado (Fernet)
Canal lateral por tiempo	<code>hmac.compare_digest</code> en todas las comparaciones de secretos
Repetición de código (replay)	Registro del último paso de tiempo consumido
Enumeración de usuarios	Mensajes uniformes + verificación bcrypt “señuelo”
CSRF	Token por sesión exigido en todos los POST
Inyección SQL	Consultas parametrizadas
Robo/fijación de sesión	Cookie firmada HttpOnly+SameSite; regeneración al autenticar

Fuera de alcance (riesgo residual, discutido en §7): *phishing*/MITM en tiempo real (TOTP es *phishable*), compromiso del dispositivo del usuario (malware), fuga de la **clave maestra** de cifrado, y ausencia de HTTPS en la demo local.

3. Diseño y arquitectura

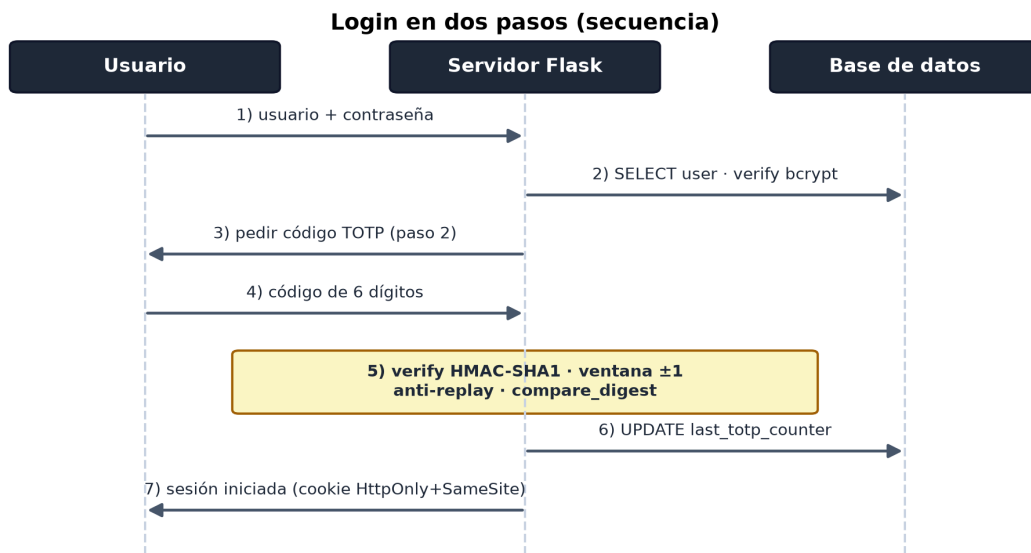


Arquitectura de componentes

El servidor Flask (`app.py`) coordina módulos con responsabilidades acotadas (principio de separación y de privilegio mínimo):

- `auth/totp.py` — HOTP/TOTP desde cero (HMAC-SHA1, truncación dinámica, `otpauth://`).
- `auth/crypto.py` — `bcrypt` para contraseñas; `SecretVault` (Fernet) para el secreto.
- `auth/db.py` — acceso a SQLite con **consultas parametrizadas**.
- `auth/ratelimit.py` — bloqueo con *backoff* y *throttle* por IP.
- `auth/recovery.py` — códigos de respaldo de un solo uso.
- `auth/accounts.py` — orquestación del registro y del login en dos pasos.
- `auth/sessions.py` — sesión segura y CSRF.

La base de datos guarda: `password_hash` (`bcrypt`), `totp_secret_enc` (secreto **cifrado**), `last_totp_counter` (anti-replay), contadores de fallos y *lockout*, hashes de códigos de respaldo y una bitácora de eventos de login.



Secuencia del login en dos pasos: contraseña (`bcrypt`) y luego TOTP con ventana ± 1 , anti-replay y comparación en tiempo constante.

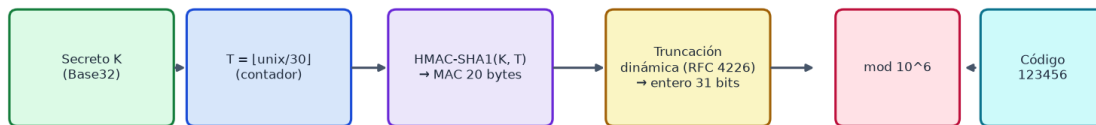
4. Implementación y decisiones de seguridad

4.1 Contraseñas: `bcrypt`

Las contraseñas se almacenan con **bcrypt** (coste 12, sal aleatoria por hash). bcrypt es deliberadamente lento, lo que encarece los ataques de diccionario *offline* si se filtrara la base de datos. Nunca se guarda ni se registra la contraseña en claro. Rechazamos contraseñas de más de 72 bytes (límite de bcrypt) de forma explícita para evitar truncamientos silenciosos.

4.2 TOTP desde cero (RFC 4226 / RFC 6238)

Cómo se genera un código TOTP (RFC 6238 / RFC 4226)



Generación de un código TOTP

Un código TOTP se obtiene aplicando HOTP al número de paso de tiempo $T = \lfloor \text{unix}/30 \rfloor$:

1. $\text{MAC} = \text{HMAC-SHA1}(\text{secreto}, T)$ con T empaquetado como entero de 8 bytes.
2. **Truncación dinámica** (RFC 4226): el último nibble del MAC indica un *offset*; se toman 4 bytes desde ahí y se descarta el bit más significativo.
3. $\text{código} = (\text{entero de 31 bits}) \bmod 10^6$, rellenado a 6 dígitos.

Generamos el secreto con un CSPRNG (`secrets` , 160 bits) y lo entregamos como QR

`otpauth://totp/IIC2531:usuario?`

`secret=...&algorithm=SHA1&digits=6&period=30` , que cualquier app autenticadora estándar interpreta.

4.3 Cifrado del secreto en reposo (corrección de la propuesta)

Nuestra propuesta original decía guardar “secretos hasheados con bcrypt”. Al implementar nos dimos cuenta de que **esto es un error conceptual**: un hash es de una sola vía, pero para verificar un código en cada login necesitamos **recalcular** $\text{HMAC}(\text{secreto}, T)$, es decir, necesitamos el secreto en claro. Un secreto TOTP no se puede *hashear*; hay que **cifrarlo** de forma reversible.

Lo ciframos con **Fernet** (AES-128-CBC + HMAC-SHA256, cifrado *autenticado*) bajo una **clave maestra** que vive fuera de la base de datos (variable de entorno). Así, si se filtra solo el archivo

`.db`, los secretos siguen protegidos. Una prueba automatizada verifica que lo almacenado es un token Fernet opaco y no el secreto.

4.4 Ventana de tolerancia y anti-replay

Aceptamos códigos del paso actual y de ± 1 paso (± 30 s) para tolerar desfase de reloj. Ampliar la ventana mejora la usabilidad pero **agranda la superficie**: cada paso extra añade códigos válidos simultáneos. Para impedir la **repetición**, guardamos el último paso de tiempo consumido por usuario (`last_totp_counter`) y rechazamos cualquier código de un paso menor o igual: un código solo sirve una vez.

4.5 Comparación en tiempo constante

Todas las comparaciones de secretos (códigos TOTP y de respaldo) usan `hmac.compare_digest`, cuyo tiempo no depende de en qué carácter difieren los valores. Una comparación ingenua (`==` con corte al primer fallo) filtra, por tiempo, cuántos caracteres iniciales son correctos, permitiendo adivinar prefijo a prefijo (§6.3).

4.6 Bloqueo por fuerza bruta (lockout)

Tras `MAX_*_ATTEMPTS` fallos (5 por defecto), la cuenta se bloquea por un tiempo que crece exponencialmente (30 s, 60 s, 120 s, ... hasta 900 s). Hay contadores separados para la contraseña y para el TOTP, y un *throttle* por IP (20 req/min) como segunda capa (defensa en profundidad). Los contadores se reinician tras un login exitoso.

4.7 Resistencia a enumeración de usuarios

Si se intenta iniciar sesión con un usuario inexistente, igualmente ejecutamos un `bcrypt.checkpw` contra un hash “señuelo” y devolvemos el **mismo** mensaje genérico (“Credenciales inválidas”) que para una contraseña incorrecta. Así el atacante no puede distinguir, ni por respuesta ni por tiempo, si el usuario existe.

4.8 Códigos de respaldo

Al enrolar se generan 10 códigos de respaldo de alta entropía (~49 bits) que se muestran **una sola vez**. Se almacenan como `sha256(sal || código)` y se verifican en tiempo constante. Cada uno se marca como usado tras emplearse (un solo uso). Como son de alta entropía, un hash

rápido (sha256) es adecuado; un hash lento (bcrypt) solo es necesario para secretos de baja entropía como las contraseñas.

4.9 Sesiones, CSRF y cabeceras

El login de dos pasos guarda el estado intermedio **en el servidor**: tras validar la contraseña la sesión queda en `pending_2fa`, y solo al validar el TOTP se promueve a sesión autenticada, **regenerando** el identificador (anti fijación de sesión). La cookie de sesión es firmada, `HttpOnly` y `SameSite=Lax` (y `Secure` bajo HTTPS). Todo POST exige un **token CSRF** por sesión. Añadimos cabeceras `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options` y `Referrer-Policy`.

5. Metodología de evaluación

Evaluamos el sistema con dos enfoques complementarios:

1. **Pruebas de correctitud** (`pytest`): vectores oficiales de los RFC y pruebas de integración de la lógica de autenticación (33 pruebas).
2. **Arnés adversarial** (`attacks/`): scripts que atacan nuestra propia implementación y **miden** el efecto de cada defensa. Todos los ataques se ejecutan localmente contra cuentas sintéticas (ética: §8).

6. Resultados

6.1 E1 — Conformidad con los RFC (correctitud e interoperabilidad)

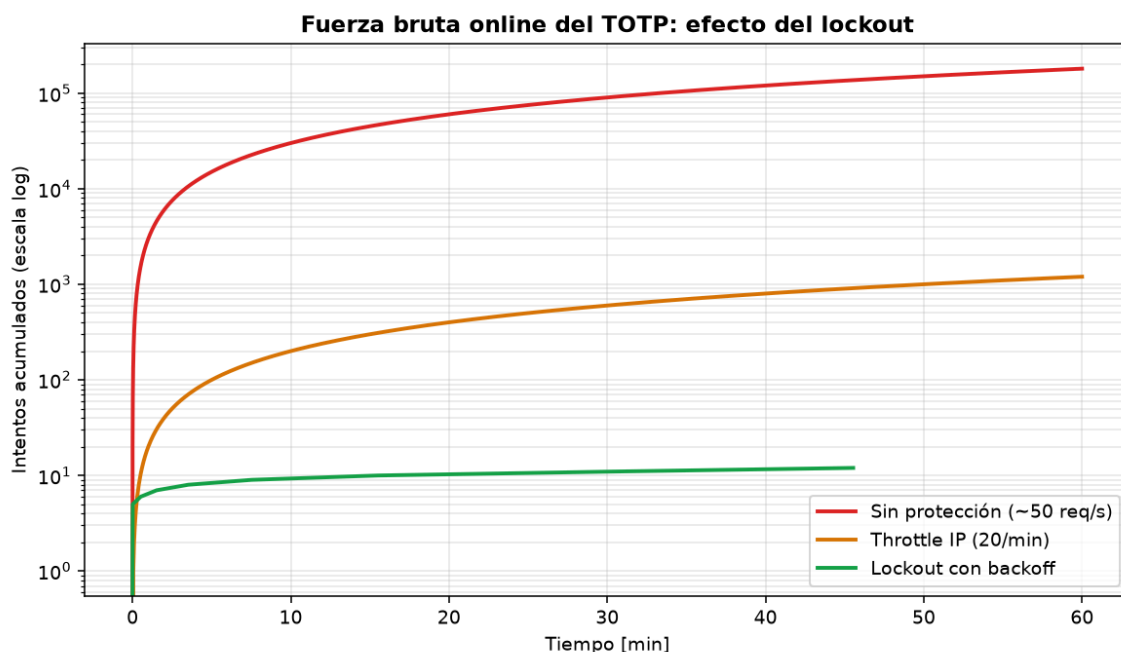
Conformidad con vectores oficiales: 16/16 ✓

RFC	Parámetro	Esperado	Obtenido	OK
RFC 4226 (HOTP)	C=0	755224	755224	✓
RFC 4226 (HOTP)	C=1	287082	287082	✓
RFC 4226 (HOTP)	C=2	359152	359152	✓
RFC 4226 (HOTP)	C=3	969429	969429	✓
RFC 4226 (HOTP)	C=4	338314	338314	✓
RFC 4226 (HOTP)	C=5	254676	254676	✓
RFC 4226 (HOTP)	C=6	287922	287922	✓
RFC 4226 (HOTP)	C=7	162583	162583	✓
RFC 4226 (HOTP)	C=8	399871	399871	✓
RFC 4226 (HOTP)	C=9	520489	520489	✓
RFC 6238 (TOTP-SHA1)	T=59	94287082	94287082	✓
RFC 6238 (TOTP-SHA1)	T=1111111109	07081804	07081804	✓
RFC 6238 (TOTP-SHA1)	T=1111111111	14050471	14050471	✓
RFC 6238 (TOTP-SHA1)	T=1234567890	89005924	89005924	✓
RFC 6238 (TOTP-SHA1)	T=2000000000	69279037	69279037	✓
RFC 6238 (TOTP-SHA1)	T=20000000000	65353130	65353130	✓

Conformidad con vectores oficiales

Nuestra implementación reproduce **16/16** vectores de prueba oficiales: los 10 de HOTP (RFC 4226, Apéndice D) y los 6 de TOTP-SHA1 (RFC 6238, Apéndice B). Esto demuestra que el algoritmo es correcto y que, por construcción, es **interoperable** con cualquier app autenticadora estándar. Lo confirmamos también enrolando el secreto en Google/Microsoft Authenticator y verificando que los códigos generados por la app son aceptados por el servidor.

6.2 E4 — Fuerza bruta online vs. bloqueo



Fuerza bruta y efecto del logout

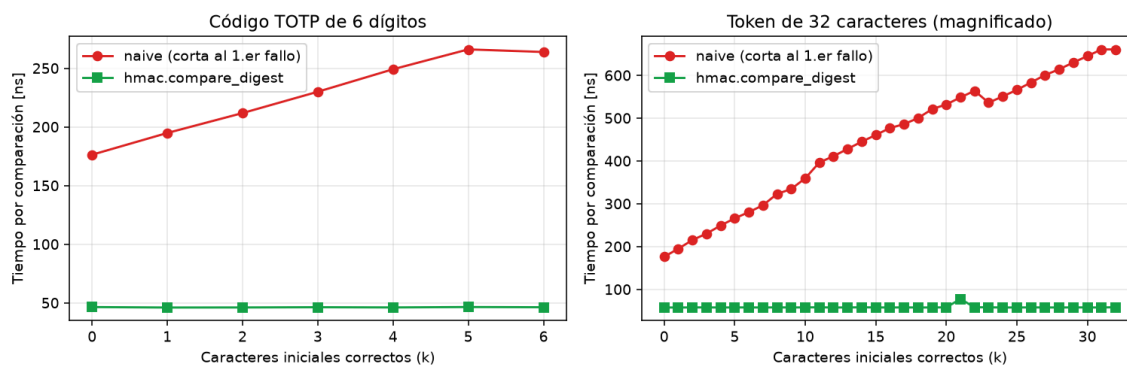
Con 6 dígitos y ventana ± 1 , la probabilidad de acertar un código al azar es $p = 3/10^6 \approx 3 \cdot 10^{-6}$. Lo decisivo es **cuántos intentos por hora** permite el sistema. Simulando con la política de bloqueo real:

Escenario	Intentos/hora	P(éxito en 1 h)	Tiempo esperado a 50%
Sin protección (~50 req/s)	180.000	0,42	~0,1 días
Solo throttle IP (20/min)	1.200	$3,6 \cdot 10^{-3}$	~8 días
Lockout con backoff	12	$3,6 \cdot 10^{-5}$	~802 días

El bloqueo reduce la tasa de adivinación en **cuatro órdenes de magnitud** y lleva el tiempo esperado de un ataque exitoso de horas a más de dos años.

6.3 E3 — Canal lateral por tiempo

Fuga por tiempo: naive vs hmac.compare_digest



Fuga por tiempo: naive vs compare_digest

Medimos el tiempo de comparación en función de cuántos caracteres iniciales coinciden. La comparación **ingenua** crece linealmente (~16 ns por carácter correcto en un token de 32 caracteres; 179 ns → 686 ns): esa pendiente es la **fuga** que permitiría descubrir el secreto carácter a carácter. `hmac.compare_digest` se mantiene **plano** (~57 ns, pendiente ≈ 0). Es una demostración directa de por qué la comparación de secretos debe ser en tiempo constante.

Honestidad metodológica: la magnitud absoluta de la fuga para un código de 6 dígitos es pequeña y, sobre una red ruidosa, difícil de explotar; el experimento de 32 caracteres la amplifica para hacerla visible. El punto es de principio: la fuga existe y `compare_digest` la elimina sin costo.

6.4 E5 — Replay

El log de `replay_demo.py` muestra que un código válido permite **un** login y que el **mismo** código, reutilizado dentro de su ventana de validez, es **rechazado** (`code=replay`), gracias al registro del último paso consumido.

6.5 E6 — Pruebas automatizadas

Las **33 pruebas** pasan: vectores RFC, validación de registro, login de dos pasos, *logout* tras N fallos, anti-replay, códigos de respaldo de un solo uso, cifrado del secreto en reposo, resistencia a enumeración y CSRF obligatorio en POST.

7. Cambios respecto de la propuesta y limitaciones

7.1 Cambios respecto de la propuesta original

- **Hash → cifrado del secreto TOTP.** El cambio más importante: la propuesta proponía *hashear* el secreto con `bcrypt`, lo cual impide verificarlo. Lo reemplazamos por cifrado autenticado (Fernet) con clave maestra fuera de la BD.
- **QR del lado del servidor.** La propuesta mencionaba `qrcode.js` en el cliente. Optamos por generar el QR en el **servidor** (librería `qrcode`, embebido como *data URI*): evita depender de JavaScript de terceros y es compatible con una CSP estricta.
- **Alcance ampliado:** agregamos códigos de respaldo, anti-replay, resistencia a enumeración, CSRF/headers y, sobre todo, el **arnés de auto-evaluación adversarial** que mide cada defensa.

7.2 Limitaciones y riesgo residual

- **TOTP es *phishable*.** Un sitio de *phishing* en tiempo real (MITM) puede pedir el código y reusarlo de inmediato en la ventana válida. El anti-replay impide reusarlo *después*, pero no detiene un proxy en vivo. La defensa real es **WebAuthn/passkeys** (ligado al origen), propuesto como trabajo futuro.
- **Gestión de la clave maestra.** La seguridad del secreto cifrado se reduce a proteger la clave maestra; si se filtra junto con la BD, se pierden ambas. En producción correspondería un

KMS/HSM. En la demo se genera una clave local de desarrollo, claramente marcada como tal.

- **Sin HTTPS / no desplegado.** La cookie `Secure` está documentada pero desactivada en HTTP local. El throttle por IP es en memoria (por proceso); en producción se usaría un store compartido.
- **Endpoint comprometido.** Ninguna 2FA protege contra malware en el dispositivo del usuario.

8. Consideraciones éticas

Todo el trabajo ofensivo (E3–E5) se ejecuta **localmente, contra cuentas sintéticas creadas por los propios scripts**. No se contactan servicios de terceros, no se recolectan credenciales reales, no hay denegación de servicio ni explotación más allá de una demostración controlada. El objetivo es *security testing* responsable de nuestro propio sistema.

9. Trabajo futuro

- **WebAuthn/passkeys** como factor resistente a *phishing*.
- Reemplazar el *throttle* en memoria por uno distribuido (Redis).
- Integración con un KMS para la clave maestra y rotación de claves.
- Soporte de TOTP SHA-256/512 y *rate-limiting* adaptativo.

10. Conclusión

Implementamos un 2FA funcional e interoperable y, más importante para un curso de seguridad, lo sometimos a un proceso adversarial que **mide** sus garantías. Aprendimos que la parte difícil de la autenticación no es el “happy path” sino los detalles que deciden si resiste un ataque: cómo se comparan los secretos, cómo se almacenan, cuántos intentos se permiten y cómo se evita la repetición. El hallazgo más valioso fue descubrir y corregir un error de seguridad en nuestra propia propuesta.

11. Cómo reproducir

```
cd auth-system
make setup      # crea venv e instala dependencias
make test       # 33 pruebas (incluye vectores RFC) ->
verde
make run        # servidor en http://127.0.0.1:5000
make experiments # genera la evidencia en
experiments/results/
make docs       # genera slides (pptx/pdf/html), informe
(docx/pdf) y guion
```

Estructura principal: `app.py` (servidor), `auth/` (módulos de seguridad), `tests/` (pruebas), `attacks/` y `experiments/` (arnés y evidencia), `docs/` (informe y figuras), `slides/` (presentación y guion).

12. Referencias

1. M'Raihi et al. **RFC 4226** — HOTP: An HMAC-Based One-Time Password Algorithm. 2005.
2. M'Raihi et al. **RFC 6238** — TOTP: Time-Based One-Time Password Algorithm. 2011.
3. Python Software Foundation. Documentación de `hmac (compare_digest)`.
4. Digital Bunker. *How do Time-based One-Time Password (TOTP) services work?*
5. OWASP. *Authentication y Password Storage Cheat Sheets*; ASVS v4.